What Airbnb discovered after launching their deep learning model for ranking

Think launching a deep learning model is hard? Try optimising it. Read how Airbnb turned post-launch failures into lessons.

JOSE PARREÑO GARCIA
AUG 11, 2025

20 2 10 Share

This week I'm picking up where we left off a few weeks ago, with Airbnb's journey into deep learning for search ranking.

In a previous post, we looked at their first steps: replacing classic ML + feature engineering with a deep neural net that actually outperformed them. That was a big win. But what happened *after* the first model launch? That's what this second paper explores.

In "**Improving Deep Learning for Airbnb Search,**" the team shares what they tried next: new architectures, debugging techniques, and lessons from A/B tests that flopped. But what I found most valuable is *how* they approached the work — not just

by importing fancy models, but by chasing down real user problems and trying to understand what the model was actually doing.

Senior Data Science Lead is a reader-supported publication. To receive new posts and support my work, consider becoming a free or paid subscriber.

Where can you find the paper?

Improving deep learning for Airbnb search (arXiv)

This paper was released in 2020 and from the abstract you understand that releasing a first deep learning model is important, but scaling it and iterating over it is a different challenge.

In this paper distillation, we will cover:

1. **Start with the user problem: “is cheaper better”?**
2. **Multiple new models were tried. None worked. ICE plots to the rescue.**
3. **The right price for the trip as part of a 2 tower architecture**
4. **Cold start problem for new hotels**

5. **Eliminating position bias**

Start with the user problem: “is cheaper better”?

From the previous referenced paper, we know Airbnb was successful in deploying deep learning for ranking. But improving deep learning required new approaches, and the starting point was understanding if cheaper is better.

Airbnb had a benefit approaching this problem, as they had a lot of experience through their initial deep learning journey, where they saw that the “series of successful ranking model launches were not only associated with an increase in bookings, **but also a reduction in the average listing price of search results** “. Aha! Something there to start investigating.

You can see how the distribution shows that there are **more travellers looking for cheaper prices than the displayed** (ie, the distribution has a higher density mass of the left hand side)

Multiple new models were tried. None worked. ICE plots to the rescue.

Airbnb took the booking price vs median result price result and undertook a series of ideas to 'force' the output of the DNN to consider that cheaper prices are better. Examples of what they tried:

- Enforcing cheaper is better through penalising prices that were higher than the median list price *(page 2, section 2.2)*
- Use price as part of the DNN learning, but make it a monotonically decreasing input to the DNN. The idea is to (page 2 & 3, section 2.3)
- Instead of adding a specific control to the DNN input, they decided to *provide the DNN with the context* that cheaper was better. To do this, they introduce a second target label, where they indicate which listing of the pair is cheaper.

However, all the above ended up in 'disaster' in terms of A/B testing results. This was curious because each of the iterations did manage to display a lower price, but still it seemed that travellers preferred (ie, booked more) the more expensive version. See? Improving deep learning wasn't that easy.

> *Understanding what the DNN was doing became key*

We all know that DNNs are an opaque blackbox. Conventional techniques miss lots of the nuances due to the great amount of non-linearities applied in the model. This is why, Airbnb turned to ICE plots.

The idea of ICE (individual conditional expectation) is to take a single example and simulate the effect of predictions when we change the range of values of an input. For example, take the listings of a search and sweep across the price range (whilst keeping all the other features constant). Repeat for 100s of searches. Plot the outputs.

(Of course this is time consuming, but sometimes you do need to go granular to understand effects.)

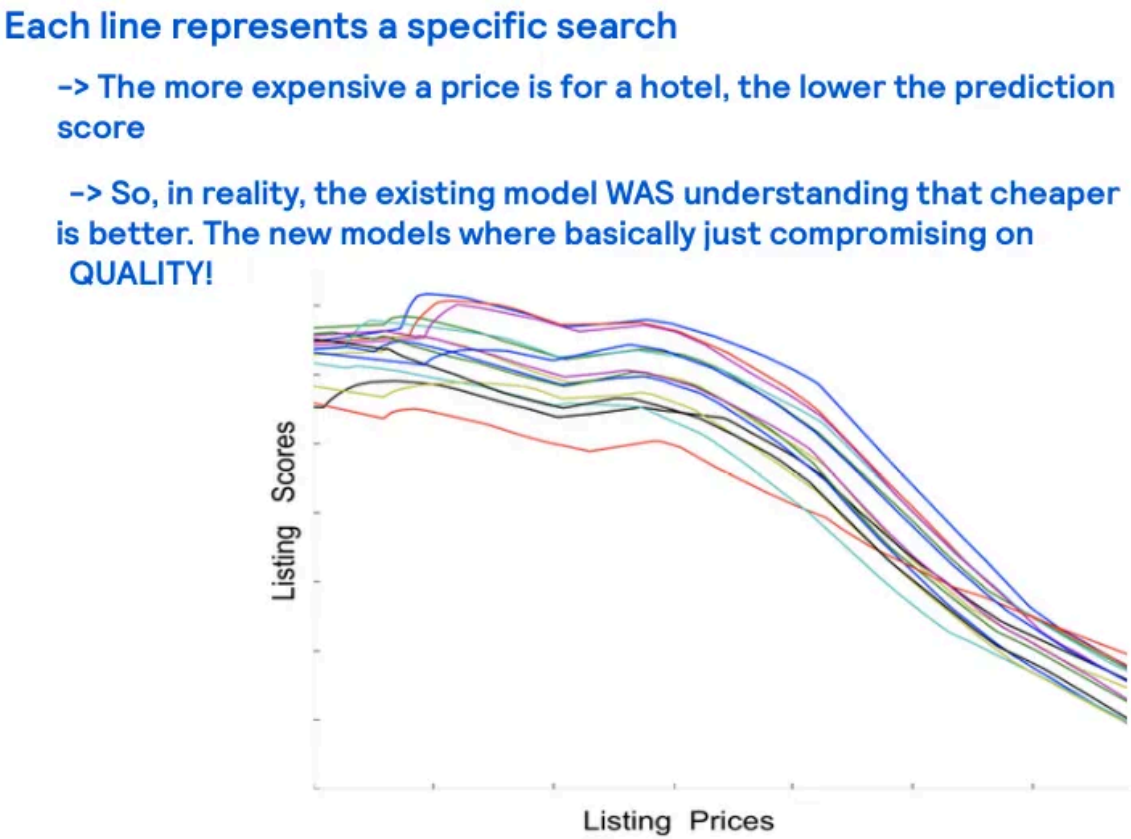


Fig. 4. ICE plot of listing scores (y-axis) vs listing prices (x-axis). Each curve represents a listing in the search result.

If the model already understood that cheaper was better, then what was the model getting wrong?

The following plot shows the difference between the median price of the list vs the price of the booking. You can clearly see big differences across different cities.

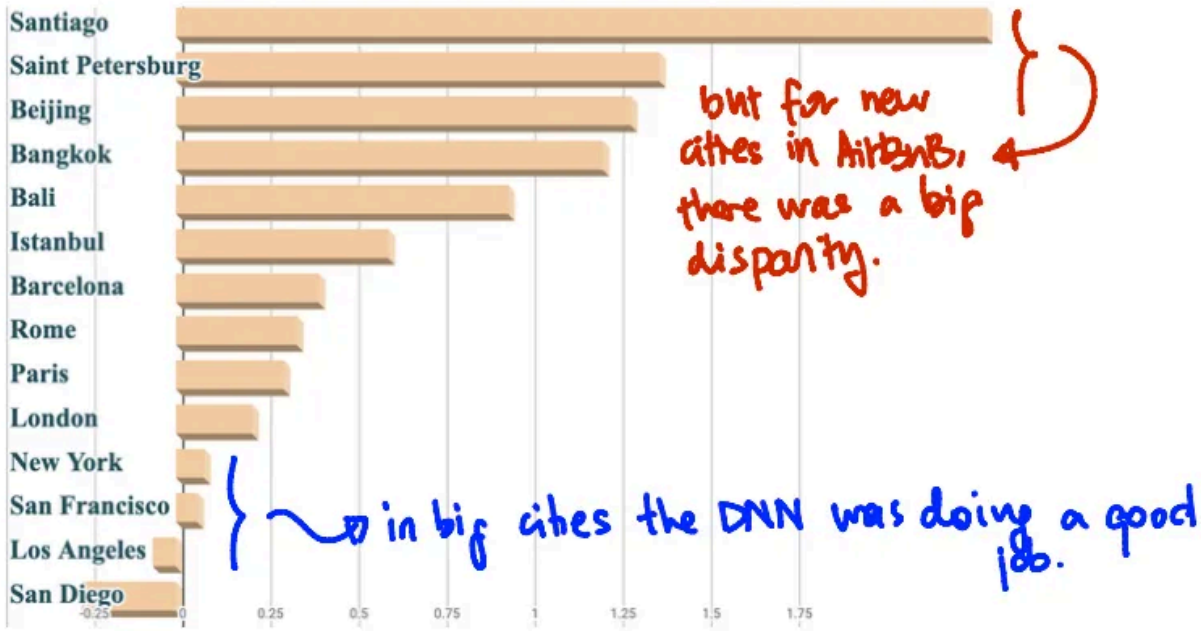


Fig. 5. Average difference between the median price of search results for a guest and the price of the booked listing, split by cities

> *"The DNN was suffering from the tyranny of the majority, focusing on price- quality tradeoffs that were tuned for the most popular locations that dominated bookings."*

Why ICE plots worked where others failed
Traditional tools like partial dependence plots (PDPs) failed Airbnb here. PDPs average the effect of a feature over all inputs — but that assumes the model treats features independently. In a DNN, that is rarely the case. Everything interacts with everything.

ICE plots, on the other hand, hold everything else constant and just vary one feature — in this case, price. That gives you a *per-listing* view of what the model is actually doing. It respects the messy, tangled relationships that deep networks learn. Perfect for a black-box model in production.

The right price for the trip as part of a 2 tower architecture

After several failed attempts to "force" the model to prefer cheaper listings, Airbnb's team realised the problem was not about pushing prices down. The model already *knew* that cheaper was better — but only in a very general sense. What it lacked was the ability to recognise the *right* price for a specific trip.

They reframed the problem: rather than penalise price directly, could the model **learn what an ideal listing looks like** for a given user and query — and then rank based on closeness to that ideal?

That idea led to the **two-tower architecture**.

Two towers, one goal

In this architecture, the model is split into two separate branches:

- One tower processes the **user and query** (location, date range, guest count, etc.)
- The other processes the **listing itself** (price, amenities, location, etc.)

Each tower produces a vector — think of it like a fingerprint that represents either the *ideal* listing (from the user/query side) or an *actual* listing (from the supply side). The model then measures how close these two vectors are — using the good old Euclidean distance.

> Booked listings should be closer to the ideal. Not-booked ones should be further away.

The loss function optimises for exactly that: pull booked listings *toward* the ideal vector, and push others *away*. Simple idea, powerful results.

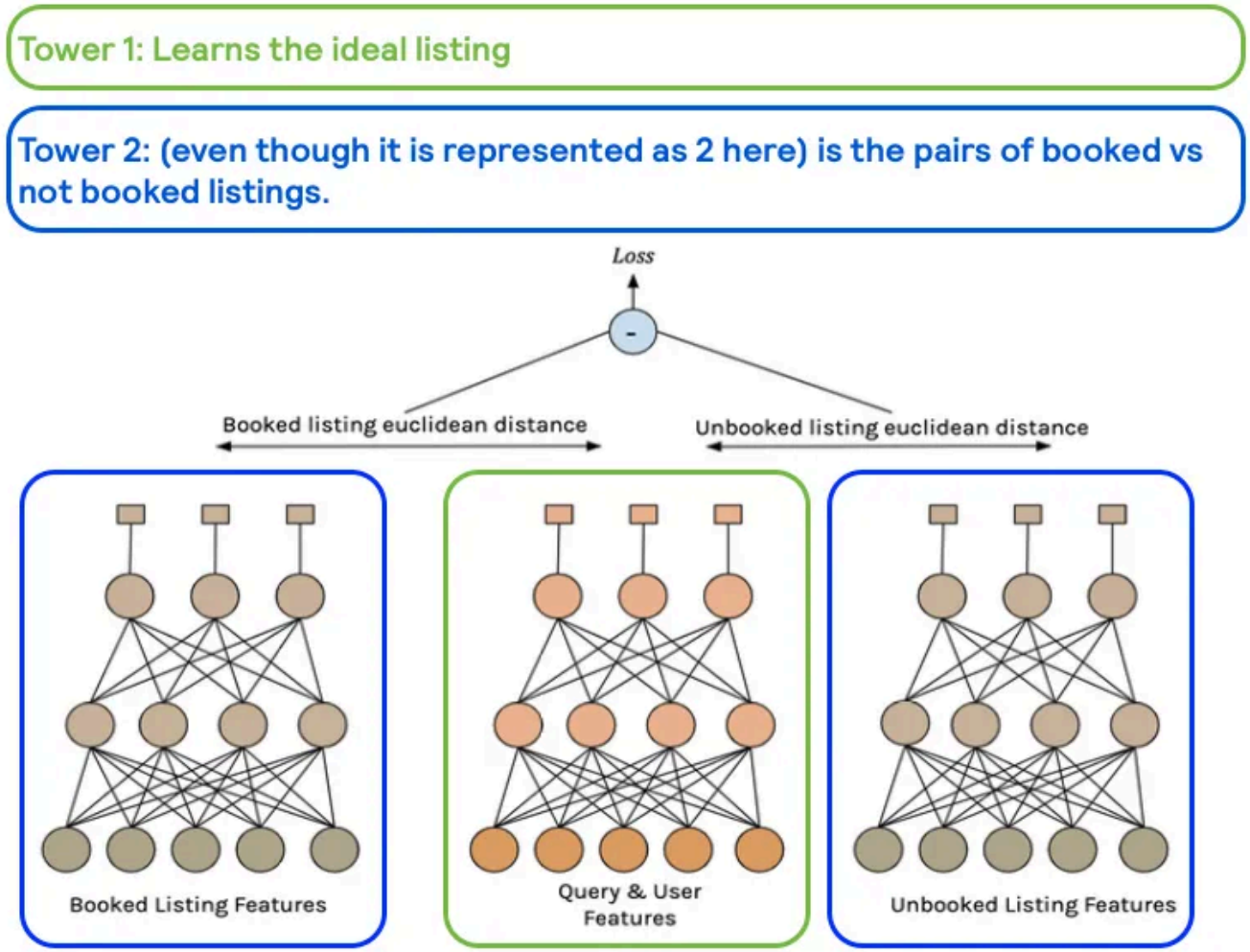


Fig. 6. Pairwise loss computation for training the two tower architecture.

What made it work?

This structure unlocked a few things:

- It allowed the model to account for **local preferences** (e.g. different price expectations in Delhi vs Paris)
- It implicitly learned the notion of the *right* price instead of hardcoding "cheaper is better"
- It enabled more flexible representations — user features and listing features no longer had to collide in the same input blob

How did the ICE plot with this new architecture?

After training the two-tower model, they re-ran the ICE plots for price. And what they found was striking: instead of price scores always going down (cheaper = better), the curves *peaked* at specific prices — a sign that the model had learned to identify a "sweet spot" for price depending on context. In other words, it was no longer blindly chasing discounts. It had learned relevance.

The model now understands:

1) There are specific price points for each search representing the "best ideal price" - see the spikes in each curve

2) Because it learns an "ideal price point", then prices cheaper than that, don't really have an effect on the prediction score. Here, the quality of the listing takes precedence.

3) Because it learns an "ideal price point", then prices higher than that, have a massive penalisation in terms of score. It doesn't matter how good the listing is in terms of quality, it is simply too expensive.

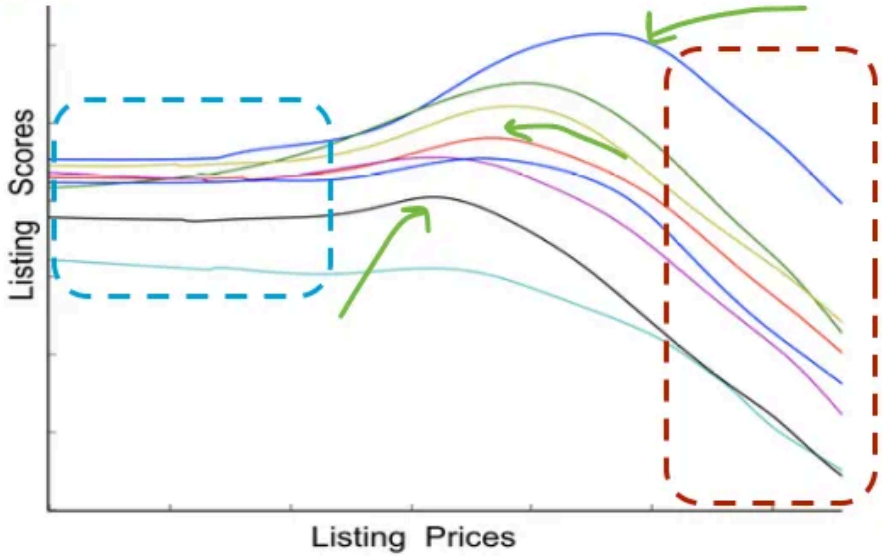


Fig. 7. ICE curves for price revisited for the two tower architecture.

Cold start problem for new hotels

One of the ways of improving deep learning is by looking at specific segments to improve. Airbnb shows in the paper how new listings where at a disadvantage compared to existing listings.

This is because, as part of their DNN, Airbnb used engagement features at the listing level. One of the implications this has is that new listings would never benefit from this information, and therefore, would be penalised at inference time.

> ***Their solution was to try to predict these user engagement features.*** *A question rises: how to evaluate if the ranking model was benefiting from this new feature?*

How do we know that our engagement estimator is good?

Well, imagine the following:

- You know your real ranking having used engagement features.
- You can score these same listings but replacing the engagement features, with the predicted ones.
- You can then compute the error in between the true rank and the predicted rank, and average across all samples.
- The estimator with the lower error, should represent the best option. In fact, en error of 0 would mean that your engagement estimator is as good as using real data.

What sort of prediction algorithm they used?

Not an ML one! They averaged the true (or real) engagement features over:

1. Geographically close (neighbours)
2. Similar guest capacity (listing attributes)
3. And using a sliding window (to account for seasonality)

As you can see, a good old heuristic is always the first best step!

What where the test results?

When launched to production, they observed an improvement of +14% in bookings of newly created listings, with an overall bookings increased by +0.38%, indicating an overall improvement in the user experience.

Eliminating position bias

Another subtle bias in Airbnb's model came from **position**. Listings that showed up higher on the page naturally got more clicks — not necessarily because they were better, but because they were easier to spot. Over time, this created a feedback loop: the model learned that high-ranking listings get booked more, which led it to keep ranking them high.

To fix this, Airbnb needed a way to **teach the model about position — without letting it depend on it** at inference time. Their solution was clever and non-intrusive.

The trick: teach it, then take it away

Here is what they did:

- **During training**, they included the listing's position as a feature — so the model could learn how position affects bookings.
- **During inference**, they always set this feature to **zero** — so every listing was evaluated "as if" it appeared in position 0. This meant that the final ranking would not depend on where the listing had appeared previously.

It is like giving the model a controlled variable: you teach it about position, but you do not let it use that information when making real decisions.

But wait — won't the model just overfit to position?

Yes. If you are not careful, the model might learn to rely too much on position during training. To avoid that, Airbnb introduced **dropout** specifically on the position feature.

This meant that, during training, the model would **randomly ignore the position input some of the time**. That forced it to learn other signals (like relevance and quality), not just fall back on "ranked higher = better".

Picking the right dropout rate

So how do you choose the best dropout value?

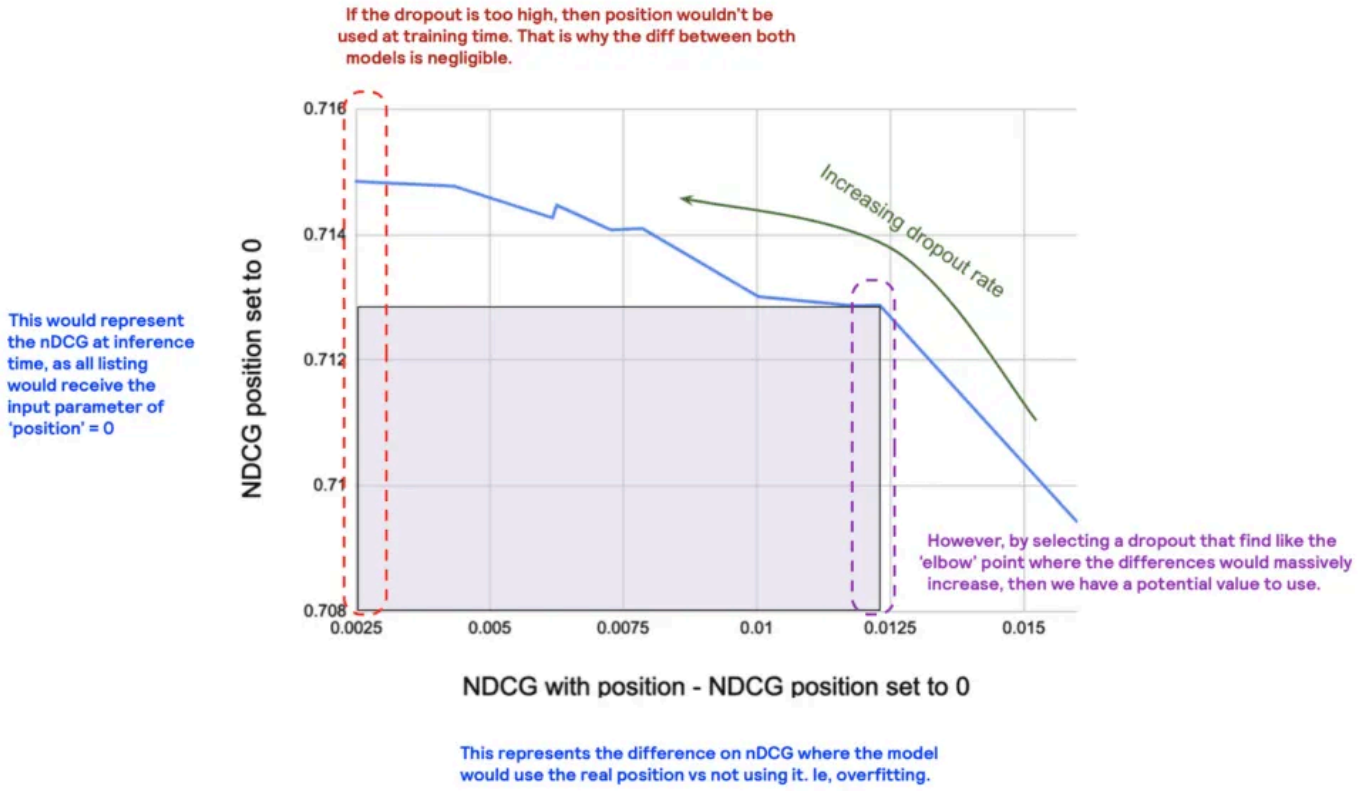
They did it by plotting two curves:

- **NDCG when position = 0**: this represents how the model behaves at inference time
- **NDCG when position is kept**: this shows how much the model is relying on position during training

Then they swept through different dropout rates. The sweet spot is the "**elbow" point** — where relying on position starts to give you a big advantage, but relevance (position = 0) has not dropped much yet. The key is to find a dropout rate where:

- You still get strong position learning (right side of the x-axis)
- But you do not sacrifice generalisation at inference time (left side, y-axis)

They ended up choosing a dropout rate around **0.15**, which balanced both goals.



Summary

Here are my biggest takeaways from Airbnb's "Improving Deep Learning for Airbnb Search":

1. Do not jump to model tweaks — start with the user problem.
Every iteration in this paper came from observing real user behaviour: a skew towards

cheaper listings, uneven performance in tail cities, cold start pain. The model followed the user, not the other way around.

2. When models get messy, turn to better diagnostics.
Partial dependence plots were not enough. ICE plots — although slower — gave Airbnb precise, interpretable insight into how price was behaving. It helped them stop guessing and start fixing.

3. Cheaper is not always better — relevance is.
Their new two-tower architecture let the model learn what the *right* listing looked like for each query. It worked not because it forced price down, but because it balanced price with context.

4. Fix cold start with predictions, not just rules.
Instead of boosting new listings with a static rule, Airbnb predicted what engagement would *have been* using local averages — a simple heuristic that drove +14% bookings for new listings.

5. Biases do not always need complex counterfactuals.
To remove position bias, they simply added position as a feature in training and zeroed it out during inference — and regularised it with dropout. Elegant, measurable, and highly effective.

Biggest learning?
Once you have a powerful model in place, further gains often come not from fancier layers — but from reframing the problem, improving your diagnostics, and embedding user understanding into the architecture itself.

Now, I want to hear from you!

- What is the hardest part *after* you have launched your first deep learning model?
- Is it figuring out what the model is really doing? Getting traction on edge cases? Or keeping performance stable across fast-changing inputs?
- Have you ever tried ICE plots, proxy features, or structured dropout like Airbnb did?
- Or maybe you faced a "cheaper is better" intuition that turned out to be… not quite true?

Drop your stories or struggles in the comments — I would love to hear how *you* make sense of deep models in production. 👇

Further reading

If you are interested in more content, here is an article capturing all my written blogs!

All my articles, one convenient place

JOSE PARREÑO GARCIA • 15 OCTOBER 2024

Read full story →

Senior Data Science Lead is a reader-supported publication. To receive new posts and support my work, consider becoming a free or paid subscriber.

20 Likes • 10 Restacks

Discussion about this post

Comments Restacks

Write a comment...

Andres Vourakis Future Proof Data Science 11 Aug 2025

Liked by Jose Parreño Garcia

Great breakdown Jose!

The hardest part in my experience is keeping performance estable in production, and throughout my years I haven't come across many problems that require DL.

LIKED (1) REPLY SHARE

1 reply by Jose Parreño Garcia

1 more comment...

© 2026 Jose Parreño Garcia · Privacy · Terms · Collection notice
Substack is the home for great culture